Multiplexer Design and Modeling Styles

Emmanuel Cano

Goal of the Exercise

The goal of this exercise was to design and simulate multiplexers using VHDL within Quartus Lite, employing three different modeling styles. The 4:1 multiplexers were designed using the Dataflow, Behavioral, and Structural modeling styles. Through these tasks, we were expected to understand how to differentiate between the three modeling styles and demonstrate how each can be used to achieve the same outcome of a desired design. Additionally, we were expected to learn how to utilize multi-bit signals, as seen in the 8-bit 4:1 MUX, and how to verify these designs through simulation in ModelSim.

Tools and Environment

- HDL: VHDL
- FPGA Toolchain: Intel Quartus Prime Lite
- Simulator: ModelSim
- Target Platform: Simulation-only (no hardware deployment)

Boolean Functions of the Multiplexers

The Boolean function for a 4:1 multiplexer is:

O = (S1’ • S0’ • A) + (S1’ • S0 • B) + (S1 • S0’ • C) + (S1 • S0 • D)

The Boolean function for a 2:1 multiplexer is:

O = (S’ • A) + (S • B)

Truth Tables

Table 1. Truth table for a 2:1 multiplexer

S	A	B	Output
0	0	0	0
0	0	1	0
0	1	0	1

0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

As seen in the 2:1 mux truth table, the output is dependent on the select input. The output is A when select = 0, and the output is B when select = 1. Therefore the 4:1 multiplexer can be simplified to the following truth table:

Table 2. Truth table for a 4:1 multiplexer

S1	S0	Output
0	0	A
0	1	B
1	0	C
1	1	D

VHDL Implementation

```
library ieee;
use ieee.std_logic_1164.all;

entity MUX4to1_Data is
    port(
        A, B, C, D : in std_logic;
        S          : in std_logic_vector(1 downto 0);
        O          : out std_logic
    );
end entity;

architecture MUX4to1_DataArch of MUX4to1_Data is
begin
    O <= ((not S(1) and not S(0)) and A) or
         ((not S(1) and     S(0)) and B) or
         ((    S(1) and not S(0)) and C) or
         ((    S(1) and     S(0)) and D);
end architecture;
```

Figure 1. *4:1 Multiplexer (Dataflow Modeling)*

Figure 1 describes the multiplexer by directly implementing the Boolean equation. It uses logical operators (AND, OR, and NOT) to connect the inputs and select lines, making the code simple and direct. This is purely combinational logic.

```
library ieee;
use ieee.std_logic_1164.all;

entity MUX4to1_Behave is
    port(
        A, B, C, D : in std_logic;
        S          : in std_logic_vector(1 downto 0);
        O          : out std_logic
    );
end entity;

architecture MUX4to1_BehaveArch of MUX4to1_Behave is
begin
    process(S, A, B, C, D)
    begin
        case S is
            when "00" => O <= A;
            when "01" => O <= B;
            when "10" => O <= C;
            when "11" => O <= D;
            when others => O <= '0';
        end case;
    end process;
end architecture;
```

Figure 2. *4:1 Multiplexer (Behavioral Modeling)*

Figure 2's design uses a process with a case statement to describe the circuit's behavior. Depending on the value of the select lines, the output is assigned one of the four inputs. This approach is more readable for multiplexers of this size than dataflow modeling. Also, no clocked logic is used in this implementation.

```
library ieee;
use ieee.std_logic_1164.all;

entity MUX4to1_Struct is
    port(
        A, B, C, D : in std_logic;
        S          : in std_logic_vector(1 downto 0);
        O          : out std_logic
    );
end entity;

architecture MUX4to1_StructArch of MUX4to1_Struct is
    component MUX2to1
        port(
            A, B, S : in std_logic;
            O       : out std_logic
        );
```

```
    end component;

    signal mux_out1, mux_out2 : std_logic;
begin
    U1 : MUX2to1 port map(A => A, B => B, S => S(0), O => mux_out1);
    U2 : MUX2to1 port map(A => C, B => D, S => S(0), O => mux_out2);
    U3 : MUX2to1 port map(A => mux_out1, B => mux_out2, S => S(1), O => O);
end architecture;
```

Figure 3. *4:1 Multiplexer (Structural Modeling)*

Figure 3 is built by instantiating three 2:1 multiplexers as components. Two are used to select between input pairs, and a third chooses between those outputs, showing how larger designs can be constructed from smaller building blocks. The 2:1 multiplexer used in this design was the class example one. This was definitely the most complicated of the 3 modeling styles, only because it isn't as direct as the other two. However, it still achieved the same results as dataflow and behavioral modeling styles.

```
library ieee;
use ieee.std_logic_1164.all;

entity MUX4to1_8bit is
    port(
        A, B, C, D : in std_logic_vector(7 downto 0);
        S          : in std_logic_vector(1 downto 0);
        O          : out std_logic_vector(7 downto 0)
    );
end entity;

architecture MUX4to1_8bitArch of MUX4to1_8bit is
begin
    process(S, A, B, C, D)
    begin
        case S is
            when "00" => O <= A;
            when "01" => O <= B;
            when "10" => O <= C;
            when "11" => O <= D;
            when others => O <= (others => '0');
        end case;
    end process;
end architecture;
```

Figure 4. *8-bit 4:1 Multiplexer*

Figure 4 design uses a behavioral modeling style (a process with a case statement) that handles multi-bit inputs (vectors). Each select line value forwards one of the 8-bit input vectors to the output, similar to the design in Figure 2. I chose a behavioral modeling style for the 8-bit mux because as I mentioned earlier, it was the one I found to be the easiest of the three styles.

Testbench Design

The same testbench was reused for all three modeling styles.

```
library ieee;
use ieee.std_logic_1164.all;

entity MUX4to1_(Style)_TB is
end entity MUX4to1_(Style)_TB;

architecture MUX4to1_(Style)_TBArch of MUX4to1_(Style)_TB is
    signal a_TB, b_TB, c_TB, d_TB, o_TB : std_logic;
    signal s_TB : std_logic_vector(1 downto 0);

    component MUX4to1_(Style) is
        port(
            A, B, C, D : in std_logic;
            S          : in std_logic_vector(1 downto 0);
            O          : out std_logic
        );
    end component;
begin
    DUT : MUX4to1_(Style)
        port map(A => a_TB, B => b_TB, C => c_TB, D => d_TB, S => s_TB, O => o_TB);

    stimulus : process
    begin
        a_TB <= '0'; b_TB <= '1'; c_TB <= '0'; d_TB <= '1';

        s_TB <= "00"; wait for 10 ns; -- Expect A
        s_TB <= "01"; wait for 10 ns; -- Expect B
        s_TB <= "10"; wait for 10 ns; -- Expect C
        s_TB <= "11"; wait for 10 ns; -- Expect D

        wait;
    end process;
end architecture MUX4to1_(Style)_TBArch;
```

Figure 5. *4:1 Multiplexer Testbench. (Style) is replaced with Data, Behave, or Struct depending on the DUT.*

Figure 5's testbench instantiates the 4:1 multiplexer as the device under test (DUT). It declares test signals for the inputs, select lines, and output. The stimulus process then applies different input and select combinations to verify that the multiplexer selects the correct input every time. Something to note is that this testbench can be used for simulation for any of the three modeling styles (dataflow, behavioral, and structural). The only difference would be the name of the entity, architecture, and component. This testbench can also be made compatible for the 8-bit 4:1 mux by using “std_logic_vector(7 downto 0);” instead of “in std_logic” for the signal and ports, as well as assigning [a, b, c, d]_TB 8-bit values (ex: ‘00001111’).

Block Diagrams

The following diagrams illustrate the logical structure of the multiplexers implemented above.

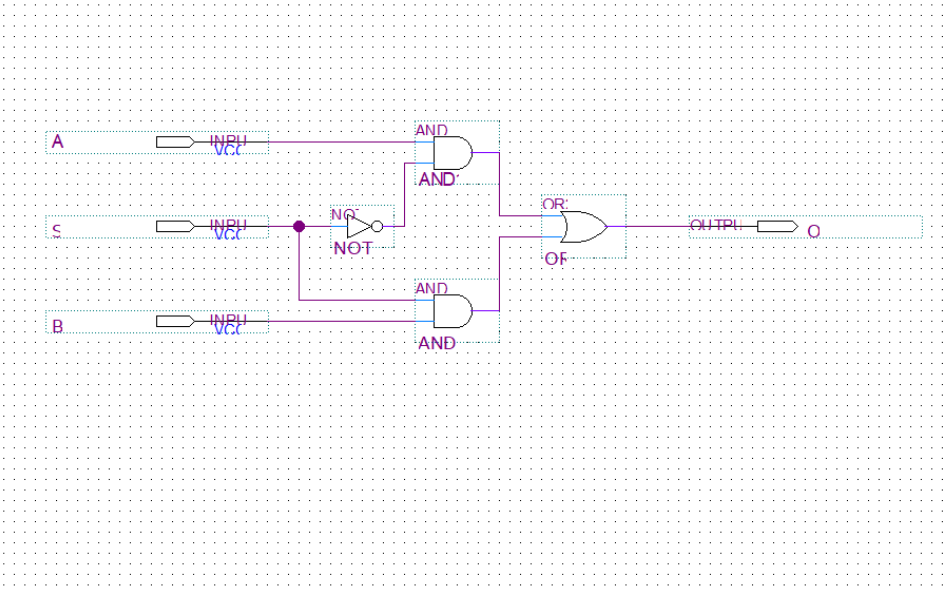

Figure 6. *Block diagram of 2:1 multiplexer using primitive gates with labeled inputs, select, and output.*

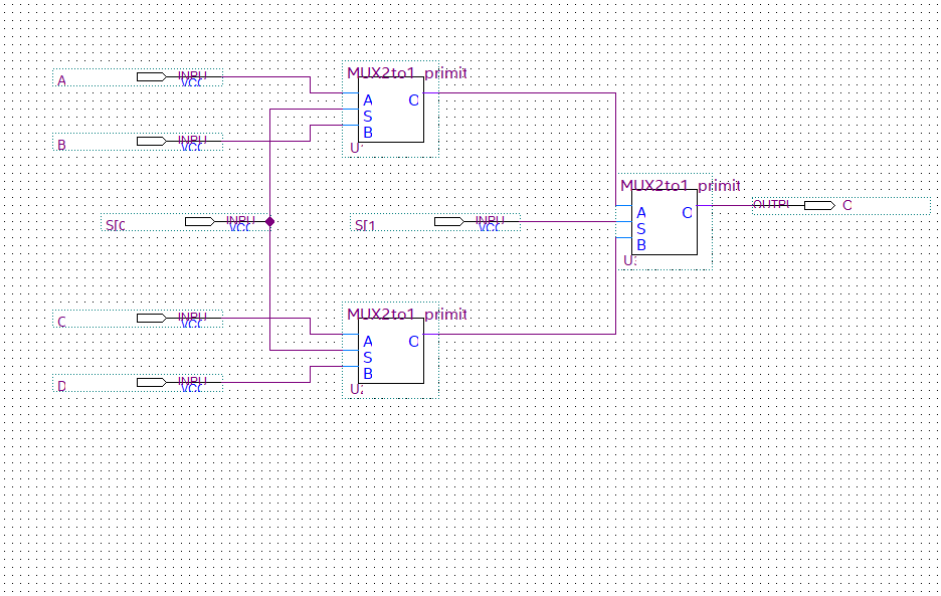

Figure 7. *Block diagram of 4:1 multiplexer with two select lines. Uses Figure 6 2:1 multiplexer symbol.*

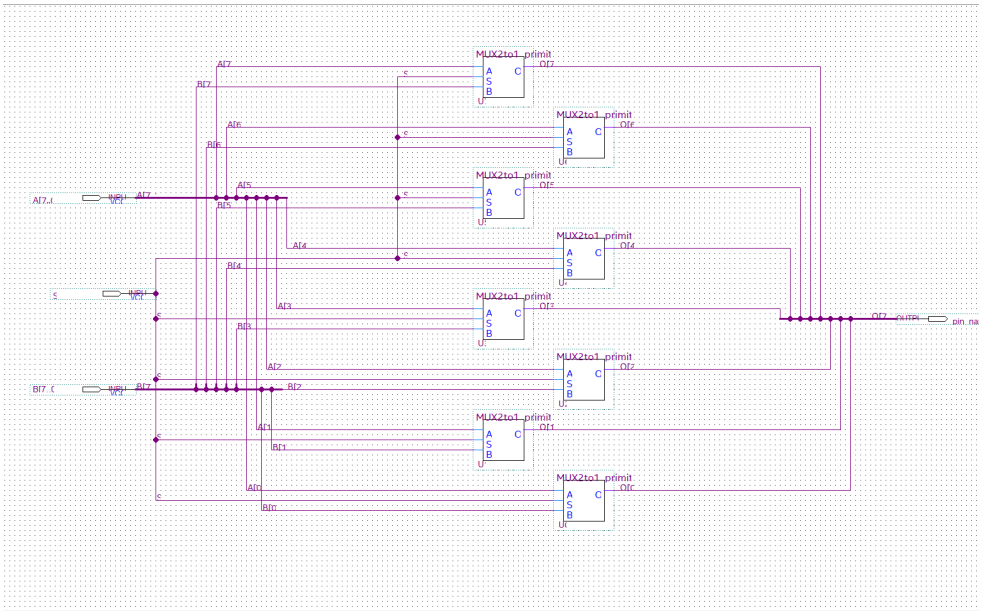

Figure 8. *Block diagram of 8-bit 2:1 multiplexer (using Figure 6 1-bit 2:1 multiplexer symbol) with labeled inputs, select, and output.*

ModelSim Simulation Results

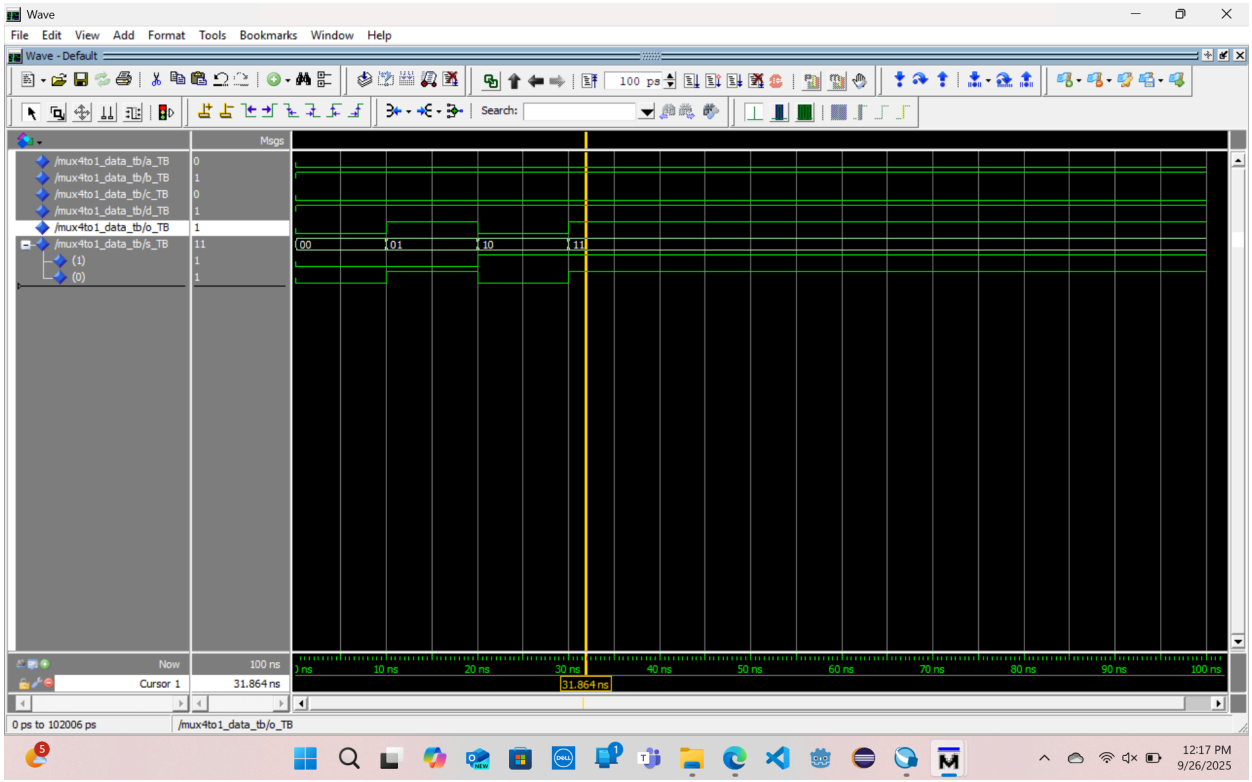

Figure 9. *Simulation results of the 4:1 multiplexer (Dataflow)*

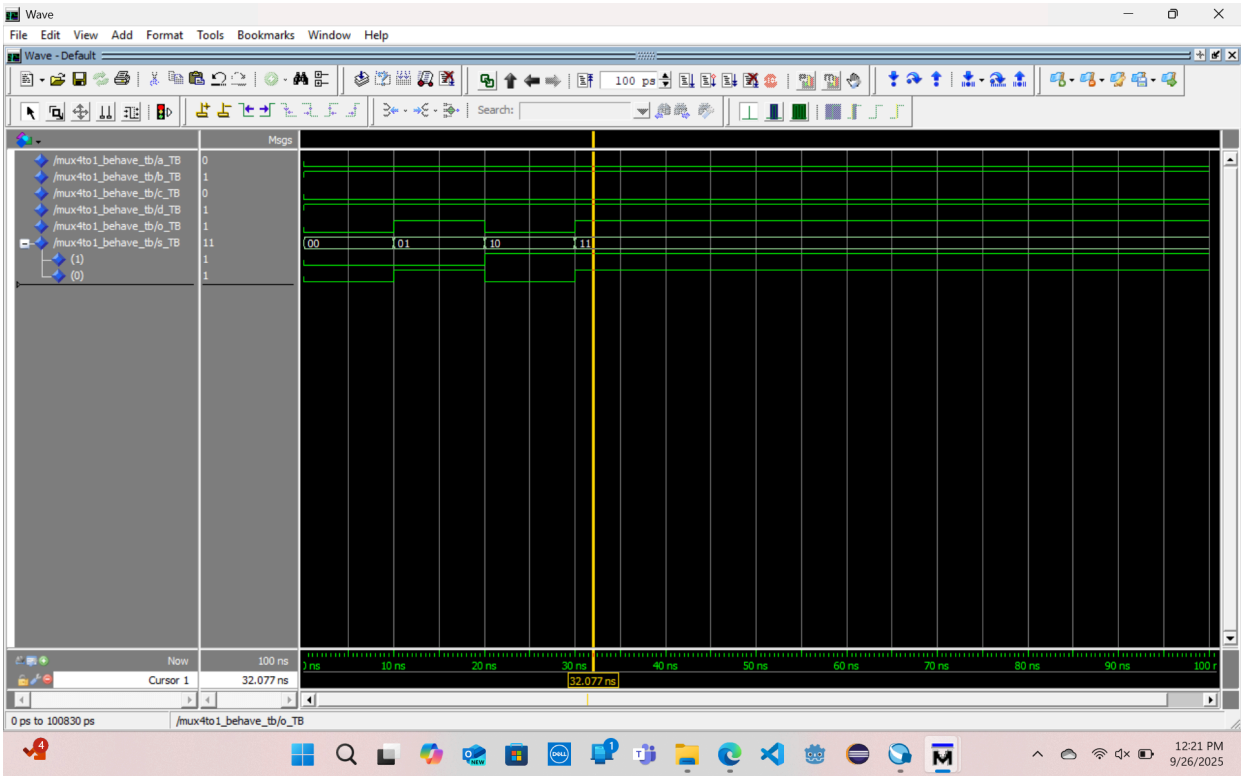

Figure 10. *Simulation results of the 4:1 multiplexer (Behavioral)*

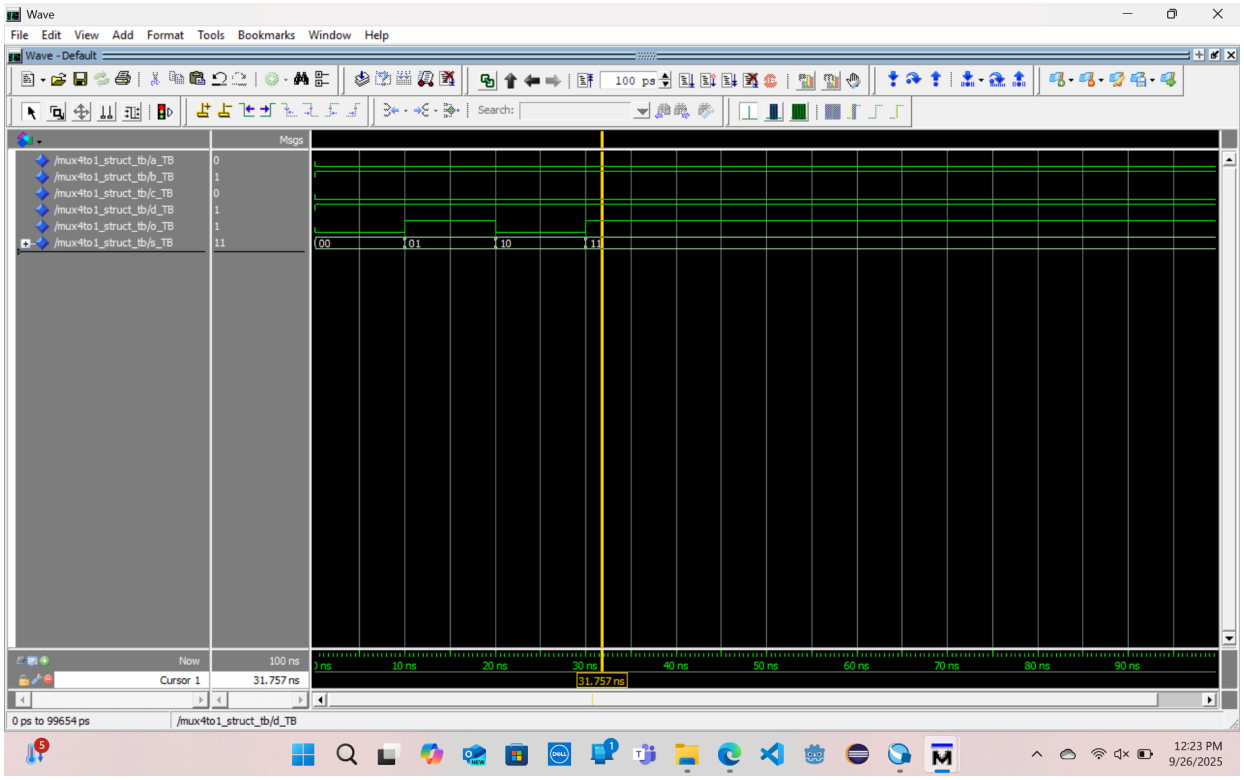

Figure 11. *Simulation results of the 4:1 multiplexer (Structural)*

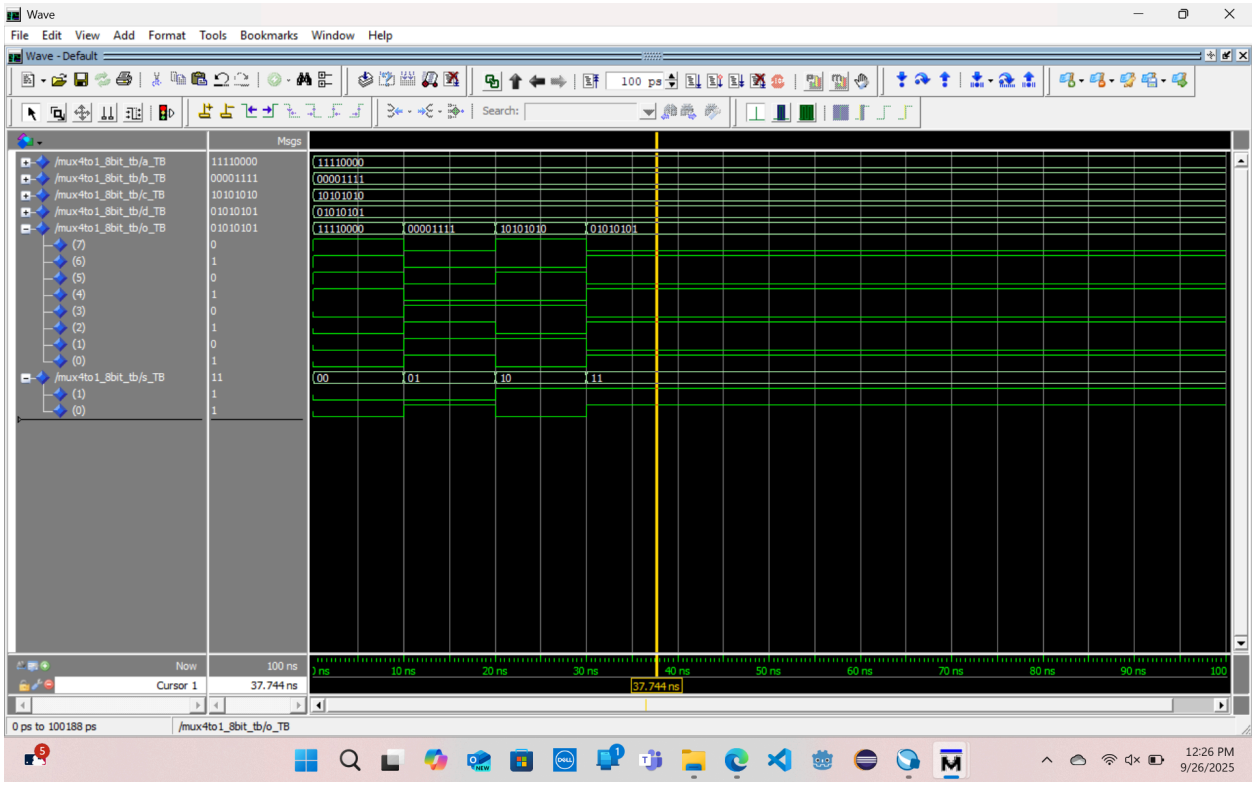

Figure 12. *Simulation of the 8-bit 4:1 multiplexer*

Conclusion

The exercise confirmed that different VHDL modeling styles can all produce the same circuit and be extended to work for more than 1 bit. Furthermore, through these different approaches, we saw the value of each style. Structural modeling helps build larger systems from smaller components, while dataflow and behavioral models allow more direct descriptions. Something I noticed is that dataflow styling resembles Boolean functions, while behavioral styling resembles truth tables. Finally, the test benches and simulations verified that the designs were correct and reliable by having the expected outputs. Also, the fact that the same testbench can be used to verify all three modeling styles demonstrates how any of the three styles can be used to create the same circuit design.